

LogInsight AI-Enterprise Intelligent Log Analysis Platform

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of
Bachelor of Technology
In
Computer Science and Engineering
School of Engineering and Sciences
Submitted by

SK.MOHAMMAD -AP23110011178
G.AKHIL BABU-AP23110011185
M.KRISHNA CHAITANYA-AP23110010966
D.ABHIRAM-AP23110011166



Under The Guidance of
Mr .RAVI TEJA

SRM University–AP Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[MAY, 2026]

LogInsightAI – Enterprise Intelligent Log Analysis Platform

Project Description

LogInsightAI is a next-generation Generative AI-powered enterprise log analysis tool built to assist DevOps professionals, security professionals and system administrators in anomaly detection, root cause analysis, and compliance monitoring of infrastructure logs. Traditionally, enterprises face challenges in keeping up with large amounts of log data due to uncorrelated disparate tools and real-time AI insights. LogInsightAI overcomes these limitations by intelligently breaking down log files, performing multi-engine anomaly detection, correlating events, predicting system issues, and reviewing logs for compliance - all under the control of an AI agent.

This tool is built on Gemini 3.1 lite preview Flash and uses the Model Context Protocol (MCP) to expose 5 tools to the LLM which it then orchestrates in an agentic workflow to process raw log files. It integrates multiple AI/ML anomaly detection tools, such as a DeepLog LSTM neural network, PyOD Isolation Forest, and Salesforce LogAI, with rule-based keyword matching and regex pattern matching to deliver enterprise-level log file analysis. Using structured data and adaptive AI insights, LogInsightAI tracks anomaly scores, severity overview, and error trends - providing holistic system health insights.

The system, developed using FastAPI, Gemini AI, FastMCP and PyTorch, integrates multiple smart components. The Multi-Engine Log Parser auto-detects log formats with Drain3, Grok, JSON and Regex parsers. The 5-Engine Anomaly Detector simultaneously uses DeepLog LSTM, PyOD Isolation Forest, Salesforce LogAI, FlashText keyword matching, and Regex pattern matching. The Event Correlator spots cascading and root causes via temporal clustering. The Predictive Failure Analyzer identifies accelerating rates of errors and vulnerable components. The Compliance Auditor checks logs for SOC2, HIPAA, PCI-DSS, ISO 27001, OWASP and GDPR compliance. Finally, Gemini 3.1 lite preview Flash translates all tool results into a report in natural language with recommendations.

By blending knowledge-based machine learning with generative AI, LogInsightAI unites disparate log analysis work to create a unified, smart, and automated incident response system. It enables faster detection, better response and continuous compliance, improving system stability, security and efficiency.

Scenario 1: DevOps Incident Investigation and Root Cause Analysis

The process begins with ops teams uploading production logs from an outage, poor performance, or other incident. The system automatically detects the log format and breaks the log into fields using 4 parsing engines (Drain3, Grok, JSON, Regex). Then, 5 parallel detection engines detect anomalies - from deviations in a sequence detected by LSTM to error patterns that match keywords. The Event Correlator then groups events within configurable time windows and identifies the initial error in each chain of events, determining the root cause. It displays the cascading failure chains, impact ranking, and an AI-generated summary, allowing teams to fix problems in seconds, not hours.

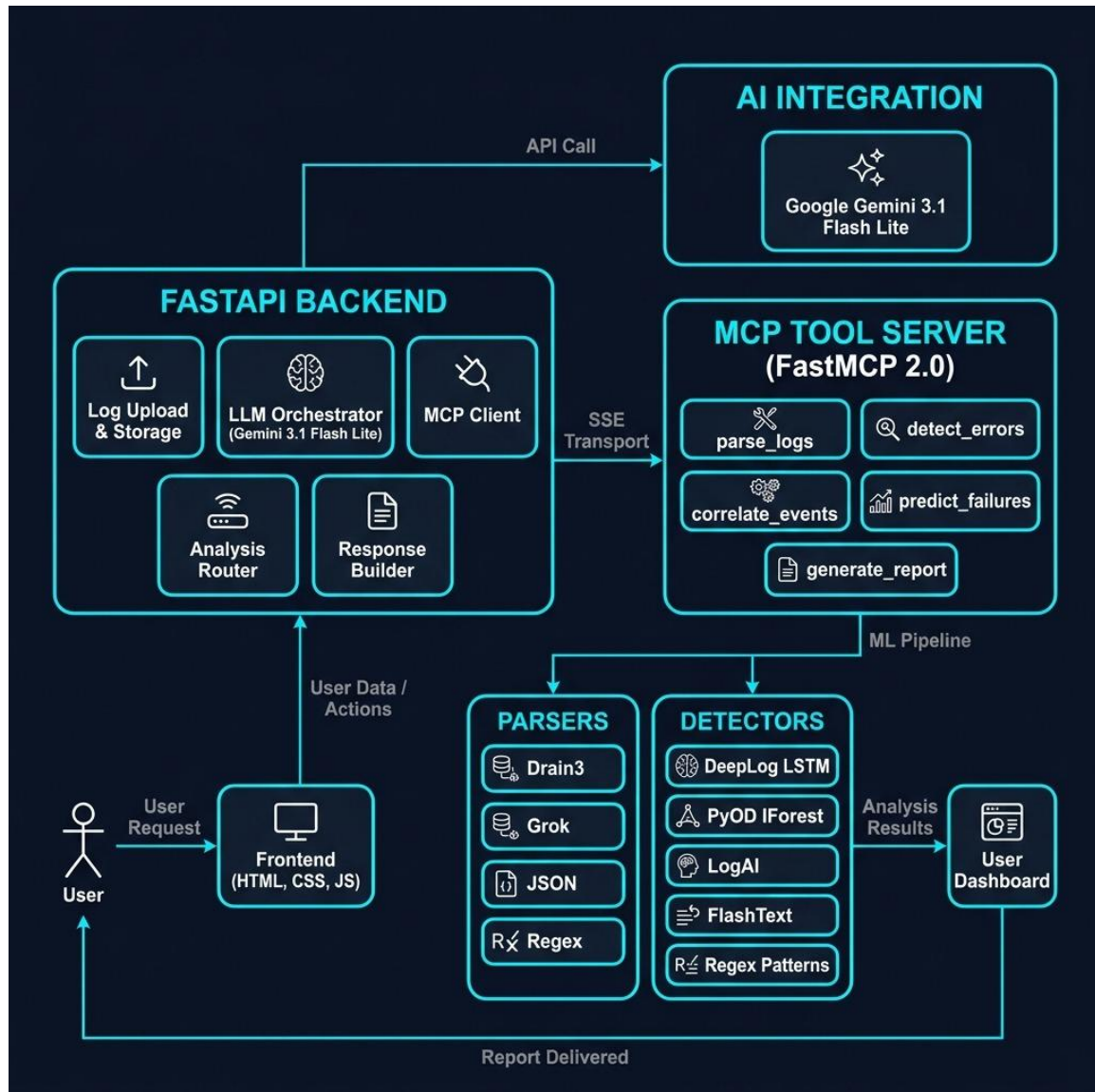
Scenario 2: Smart Anomaly Detection and Failure Prediction

Using the Analysis Dashboard, users can initiate a comprehensive AI analysis of log data. Gemini 3.1 lite preview Flash agents the MCP tools, selecting which tools to run and in which sequence, based on the log data. The DeepLog LSTM model learns the normal patterns of log sequences and identifies anomalies, while PyOD Isolation Forest identifies statistical outliers using line length as a feature. The Predictive Failure Analyzer, using the observed error trends and the error risk for each component, predicts the acceleration of errors and the risk for components with a confidence score. This approach provides timely and contextualized alarms to the team, avoiding outages with proactive monitoring and AI-based error trend analysis.

Scenario 3: Compliance Auditing and Security Reports

When organizations perform maintenance, LogInsightAI scans logs for non-compliance and security issues. The Compliance Auditor checks logs against 6 key standards—SOC2, HIPAA, PCI-DSS, ISO 27001, OWASP, GDPR - for exposed PII (emails, SSNs, credit cards, API keys), audit logs, and security events. When it identifies SQL injection, failed logins or leaks of sensitive information, it produces a full compliance report with letter grades (A-F), recommendations and findings for each framework.

Architecture Overview



LogInsightAI is an AI-powered, modular enterprise log analyzer to ingest logs, identify anomalies, correlate events, and generate compliance reports. It uses a FastAPI-based backend with Google's Gemini 3.1 lite preview Flash model connected via the Model Context Protocol (MCP) to orchestrate 5 focused analysis tools in an agentic manner.

The platform integrates deep learning (DeepLog LSTM), statistical ML (PyOD Isolation Forest), ML pipelines (Salesforce LogAI) and rule-based detection (FlashText, Regex) to provide a holistic analysis capability.

Structured log analysis allows for the detection of anomalies, error severity, and trend analysis, while smart correlation modules detect cascading anomalies and root causes across system components.

The system uses a three-tiered architecture:

- Client (Vanilla JS) - Clinical Precision UI with drag-drop upload, real-time analysis dashboard, markdown summaries by AI
- Backend (FastAPI + Uvicorn on port 8000) - Routes, LLM Orchestrator (Gemini 3.1 lite preview) and MCP client
- MCP Server (FastMCP + SSE on port 8001) - 5 tools: parse_logs, detect_errors, correlate_events, predict_failures, generate_report

Data Flow:

- User uploads a .log file → Backend saves it in uploads/
- User clicks "Run Analysis" → Backend forwards log to LLM Orchestrator
- Gemini 3.1 lite preview Flash determines MCP tools to use and order
- LLM Orchestrator calls tool on MCP Server via HTTP Server-Sent-Event (SSE)
- MCP Server runs a tool (parse/detect/correlate/predict/report)
- Results return: MCP → LLM → Backend → Frontend
- Frontend displays structured results + markdown summary

Core Technologies

FastAPI: Fast asynchronous web framework for routing, CORS, file uploads, and bridging between the frontend, Gemini AI, MCP server and file storage.

Uvicorn: ASGI server with hot-reloading to serve the FastAPI backend and static files for the frontend.

Gemini 3.1 lite preview Flash (Vertex AI): Main generative AI model that agentially controls the analysis workflow, determining which MCP tools to use, and how to interpret and report results.

FastMCP + MCP SDK: Implements the Model Context Protocol server exposing 5 specialized tools via SSE (Server-Sent Events).

PyTorch: Powers the DeepLog LSTM neural network for self-supervised anomaly detection.

PyOD: Provides Isolation Forest algorithm for statistical anomaly detection using engineered features.

Salesforce LogAI: ML-powered log analysis pipeline combining Drain3 template mining with K-Means clustering.

Drain3 (IBM Research): Automatic log template mining algorithm that groups similar log lines into templates with wildcards.

FlashText: Ultra-fast Aho-Corasick keyword extraction scanning 100+ error/warning/security terms.

Vanilla JS Frontend: Single-page application with Clinical Precision design using Outfit + DM Mono fonts, Vanta.js, AOS, and Typed.js.

Component-Wise Architecture

Component	Description
Frontend UI	Developed with Vanilla JavaScript, HTML, CSS using Clinical Precision theme. Provides file upload, log panel, severity plots, engine stats and AI report.
FastAPI Backend	Routes requests (/api/logs/upload, /api/analysis/analyze, /api/logs/list), cross-origin request sharing (CORS) and file storage. Integrates Frontend, LLM Orchestrator and MCP Server
LLM Orchestrator	Intelligent agent using Gemini 3.1 lite preview Flash. Determines which MCP tools to run, processes the results and creates reports.
MCP Server (FastMCP)	Port 8001 with SSE. Registers 5 tools: parse_logs, detect_errors, correlate_events, predict_failures, generate_report.
Log Parsing Engine	Co-ordinates 4 parsers (Drain3, Grok, JSON, Regex) to automatically identify formats, fields, templates, or summaries.
Anomaly Detection Engine	Concurrently runs 5 engines: DeepLog LSTM, PyOD Isolation Forest, Salesforce LogAI, FlashText, Regex Patterns.
Event Correlation Engine	Time clustering, root cause analysis, cascading incident analysis, component impact analysis
Predictive Failure Engine	Trend analysis, risk scoring of components; LOW/MEDIUM/HIGH/CRITICAL predictions.
Compliance & Audit Engine	Checks for SOC2/HIPAA/PCI-DSS/ISO 27001/OWASP/GDPR. Data Masking, audit trails, compliance ratings.

Pre-requisites

Google Cloud Project with Vertex AI API activated:

<https://cloud.google.com/vertex-ai>

Python 3.10+ with pip package manager

FastAPI & Uvicorn: <https://fastapi.tiangolo.com/>

PyTorch (optional for DeepLog LSTM)

MCP SDK & FastMCP: <https://modelcontextprotocol.io/>

PyOD Library: <https://pyod.readthedocs.io/>

Drain3 Library: <https://github.com/logpai/Drain3> Project Workflow

1. Setting up Gemini AI & MCP

Activity 1.1: Create Google Cloud account and activate Vertex AI API to use Gemini 3.1 lite preview Flash.

Activity 1.2: Set up API keys/environment variables and perform test request-response interactions with the Gemini model.

Activity 1.3: Conceptualize FastMCP server architecture, register 5 expert services.

2. Core Functionality Development

Activity 2.1: Plan FastAPI server structure for log uploads, files, and analysis.

Activity 2.2: Develop /api/logs/upload, /api/analysis/analyze, /api/analysis/parse, /api/analysis/detect.

Activity 2.3: Create LLM Orchestrator with agentic tool making loop for Gemini 3.1 lite preview Flash.

3. Setting Up the Backend (FastAPI & MCP Server)

Activity 3.1: Set up FastAPI routes for log file manipulation and log analysis.

Activity 3.2: Structuring backend - routes/, services/, models/.

Activity 3.3: Set up CORS and serve static files for frontend.

Activity 3.4: Set up MCP Server with 5 tool registrations, use SSE transport

4. UI Development (HTML/CSS/JS)

Activity 4.1: Create single-page application with landing page, dashboard page.

Activity 4.2: Format with Clinical Precision design system in CSS Custom Properties.

Activity 4.3: Add JavaScript for dynamic content updates - file upload, analysis and results.

5. Testing & Optimization

Activity 5.1: Experiment on 16 sample log files from different formats.

Activity 5.2: Test all 5 engines with benchmark data from LogHub.

Activity 5.3: Optimize detection accuracy, LSTM hyperparameters, Isolation Forest hyperparameters

Activity 5.4: Implement fallback for missing PyTorch, graceful degradation.

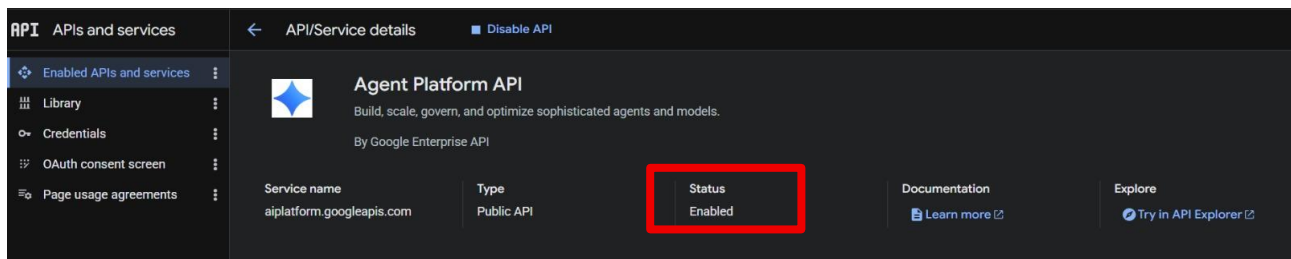
MILESTONE 1: Gemini AI & MCP Initialization

In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power LogInsight AI. The platform relies on Gemini 3.1 lite preview Flash—Google’s efficient generative model accessed via Vertex AI—to orchestrate 5 specialized MCP tools in an agentic workflow for comprehensive log analysis. This milestone ensures that the development environment is equipped with authenticated access to Google’s Gemini API and that the Model Context Protocol server is properly configured with all tool registrations.

Proper completion of this setup is essential, as it enables all AI-driven functionalities—including agentic tool orchestration, multi-engine analysis, and natural language report generation—to function reliably throughout the application.

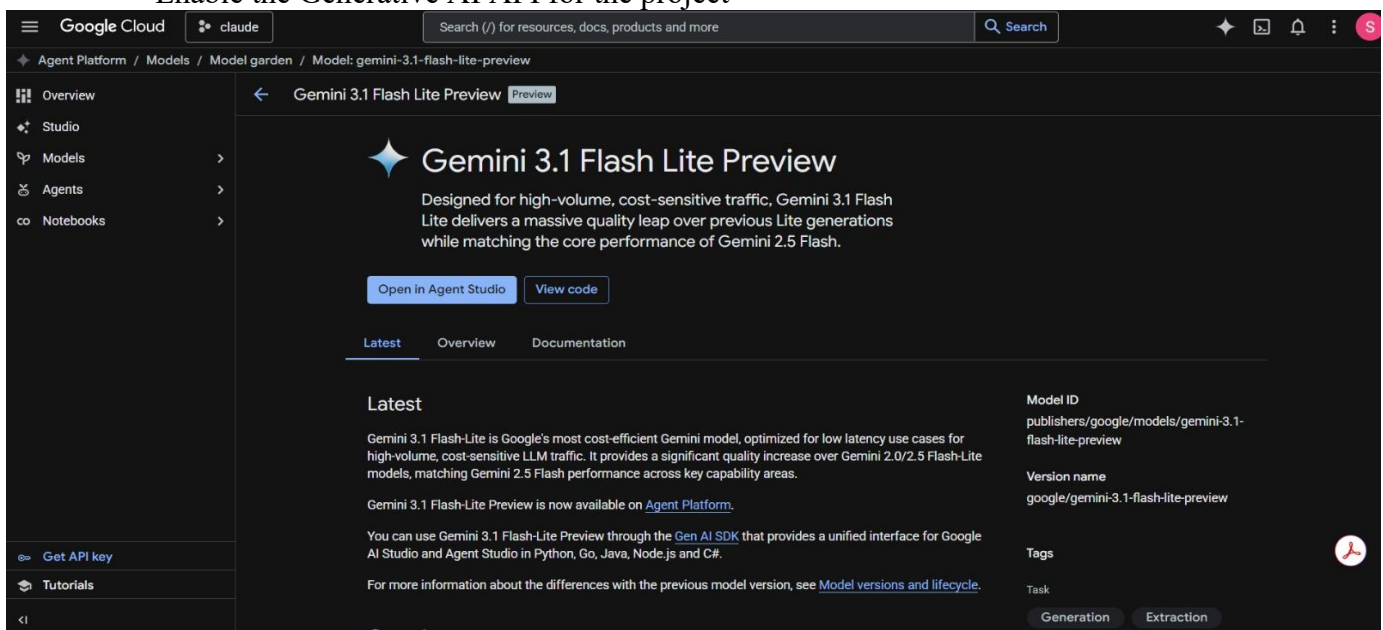
Activity 1.1: Set up Google Cloud account and enable Vertex AI API

Sign in or create an account at <https://console.cloud.google.com>. Enable the Vertex AI



API, configure authentication, and test access to the Gemini 3.1 lite preview Flash model.

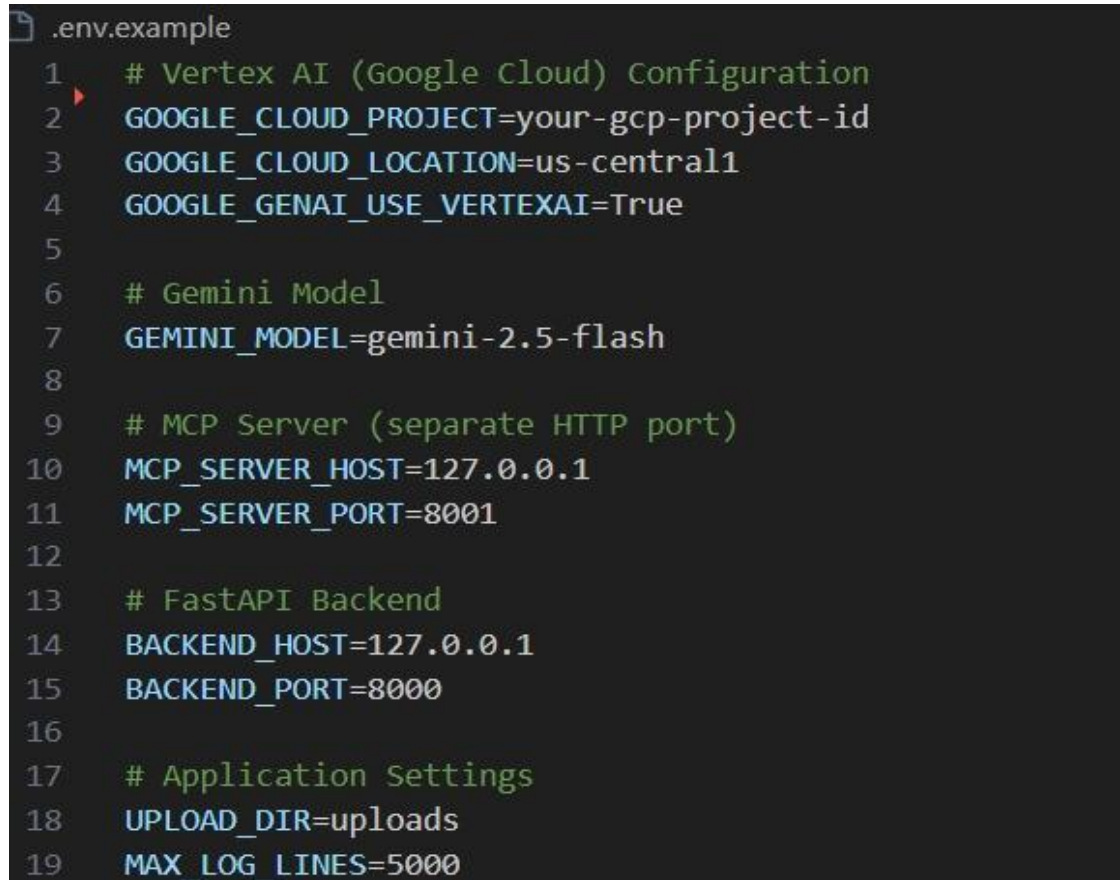
- Accept all terms and conditions and navigate to the Vertex AI section
- Enable the Generative AI API for the project



Activity 1.2: Configure access and environment variables

Create the .env configuration file with required environment variables:

```
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
GOOGLE_CLOUD_LOCATION=us-central1
GOOGLE_GENAI_USE_VERTEXAI=True
GEMINI_MODEL=gemini-2.5-flash
MCP_SERVER_HOST=127.0.0.1
MCP_SERVER_PORT=8001
BACKEND_HOST=127.0.0.1
BACKEND_PORT=8000
UPLOAD_DIR=uploads
MAX_LOG_LINES=5000
```



```
.env.example
1  # Vertex AI (Google Cloud) Configuration
2  GOOGLE_CLOUD_PROJECT=your-gcp-project-id
3  GOOGLE_CLOUD_LOCATION=us-central1
4  GOOGLE_GENAI_USE_VERTEXAI=True
5
6  # Gemini Model
7  GEMINI_MODEL=gemini-2.5-flash
8
9  # MCP Server (separate HTTP port)
10 MCP_SERVER_HOST=127.0.0.1
11 MCP_SERVER_PORT=8001
12
13 # FastAPI Backend
14 BACKEND_HOST=127.0.0.1
15 BACKEND_PORT=8000
16
17 # Application Settings
18 UPLOAD_DIR=uploads
19 MAX_LOG_LINES=5000
```

Activity 1.3: Validate Gemini API and MCP connectivity

Make a sample prompt call using Python to confirm Gemini integration.

```
import vertexai
from vertexai.generative_models import GenerativeModel

# ♦ Replace with your values
PROJECT_ID = "your-project-id"
LOCATION = "us-central1" # or asia-south1 (closer to India)

# Initialize Vertex AI
vertexai.init(project=PROJECT_ID, location=LOCATION)

# Load Gemini model
model = GenerativeModel("gemini-1.5-flash")

# Generate response
response = model.generate_content("Explain Artificial Intelligence in one sentence.")

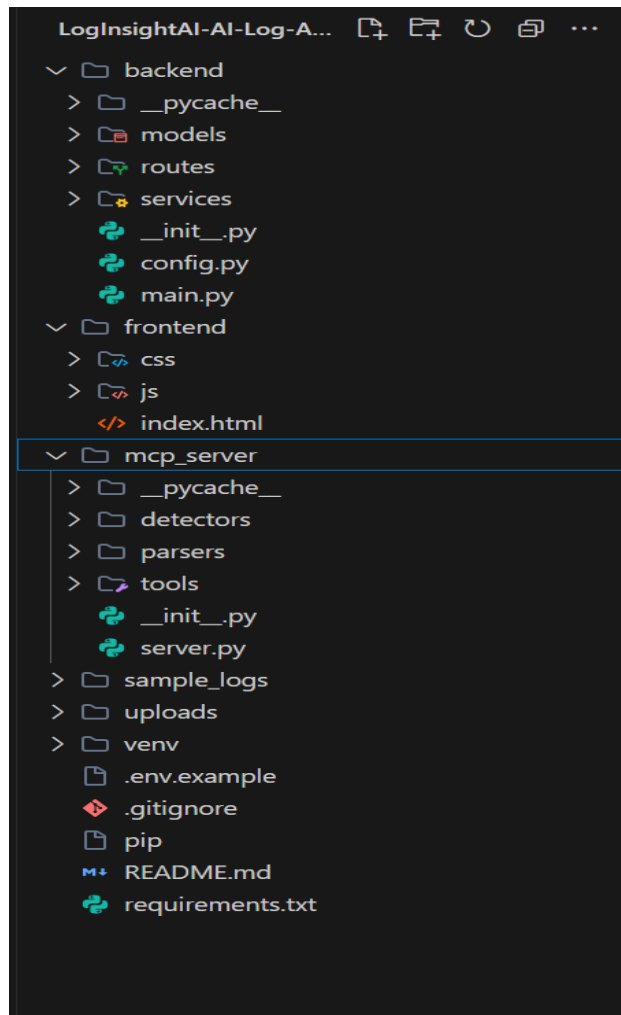
print("Response:")
print(response.text)
```

Test the FastMCP server startup .

```
C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main>cd mcp_server
C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main\mcp_server>python server.py
[MCP] LogInsight MCP Server starting on http://127.0.0.1:8001
[MCP] Registered 5 tools: parse_logs, detect_errors, correlate_events, predict_failures, generate_report
INFO: Started server process [20356]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO: 127.0.0.1:64279 - "GET /sse HTTP/1.1" 200 OK
INFO: 127.0.0.1:64280 - "POST /messages/?session_id=2a157f77164140779c75e324f7bef676 HTTP/1.1" 202 Accepted
```

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core backend intelligence of LogInsightAI using the FastAPI framework and Google's Gemini 3.1 lite preview model with MCP integration. The primary goal is to develop a robust and modular system capable of accepting log file uploads, intelligently parsing log structures, running multi-engine anomaly detection, and returning comprehensive analysis results. The backend handles file management, MCP client communication, LLM orchestration, and structured response formatting. All core logic is organized into FastAPI routes, service modules, and Pydantic schema definitions.



Folder Structure

Activity 2.1: FastAPI Initialization and import libraries

- Import all required libraries for the project.

FastAPI app initialization with lifespan management.

```
# Create FastAPI app
app = FastAPI(
    title="LogInsight AI",
    description="Intelligent Log Analyzer",
    version="1.0.0",
    lifespan=lifespan,
)

# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Ctrl+L for Agent
# Include API routers
app.include_router(logs.router)
app.include_router(analysis.router)
```

Load .env environment using config module.

```
# Load .env file
load_dotenv()

# Project paths
PROJECT_ROOT = Path(__file__).parent.parent
UPLOAD_DIR = PROJECT_ROOT / os.getenv("UPLOAD_DIR", "uploads")
SAMPLE_LOGS_DIR = PROJECT_ROOT / "sample_logs"
FRONTEND_DIR = PROJECT_ROOT / "frontend"

# Ensure upload directory exists
UPLOAD_DIR.mkdir(exist_ok=True)

# Google Cloud API Key (Vertex AI)
GOOGLE_CLOUD_API_KEY = os.getenv("GOOGLE_CLOUD_API_KEY", "")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")

# MCP Server Configuration
MCP_SERVER_HOST = os.getenv("MCP_SERVER_HOST", "127.0.0.1")
MCP_SERVER_PORT = int(os.getenv("MCP_SERVER_PORT", "8001"))
MCP_SERVER_URL = f"http://{MCP_SERVER_HOST}:{MCP_SERVER_PORT}"
```

Activity 2.2: Core Functionality - MCP Tool Implementations parse_logs:

Coordinates 4 different log parsers (Drain3, Grok, JSON, Regex) to auto detect the log format and extract templates and statistical metrics.

The `parse_logs` tool uses Drain3 template mining to extract log patterns with wildcards.

```
def parse_logs(log_content: str, log_format: str = "auto") -> str:
    """
    Parse system log content to extract structured information.

    This tool analyzes raw log text using multiple parsing engines:
    - Drain3 for template mining (clustering similar log messages)
    - Grok patterns for structured field extraction
    - JSON parser for JSON-formatted logs
    - Regex fallback for timestamp, level, and component extraction

    Args:
        log_content: The raw log file content as a string.
        log_format: Expected log format. Use "auto" for automatic detection,
                   or specify: "syslog", "apache_access", "apache_error",
                   "hdfs", "zookeeper", "json", "general_timestamp".

    Returns:
        JSON string with parsed results including templates, entries, and statistics.
    """
    log_lines = log_content.strip().split("\n")
    if not log_lines:
        return json.dumps({"error": "No log content provided", "entries": []})

    results: dict[str, Any] = {
        "total_lines": len(log_lines),
        "format_detected": None,
    }

    # Step 1: Check if JSON format
    json_parser = JSONParser()
    if log_format in ("auto", "json") and json_parser.is_json_log(log_lines):
        json_result = json_parser.parse(log_lines)
        results["json_parsing"] = json_result
        results["format_detected"] = "json"
    else:
```

`detect_errors(log_content, sensitivity="medium")`: Runs 5 detection engines in parallel.

```
class DrainParser:
    """Parse logs using Drain3 template mining algorithm."""

    def __init__(self, sim_th: float = 0.4, depth: int = 4):
        self.sim_th = sim_th
        self.depth = depth
        self._miner = None

    def _get_miner(self):
        """Lazy-initialize the TemplateMiner."""
        if self._miner is None:
            from drain3 import TemplateMiner
            from drain3.template_miner_config import TemplateMinerConfig

            config = TemplateMinerConfig()
            config.drain_sim_th = self.sim_th
            config.drain_depth = self.depth
            config.masking = [
                {"regex_pattern": r"(\d+\.\d+\.\d+\.\d+)", "mask_with": "<IP>"},
                {"regex_pattern": r"(blk_-\d+)", "mask_with": "<BLOCK>"},
                {"regex_pattern": r"(/[\\w./\\-_]+)", "mask_with": "<PATH>"},
                {"regex_pattern": r"(\d{4}-\d{2}-\d{2}[T ]\d{2}:\d{2}:\d{2})", "mask_with": "<TIMESTAMP>"},
                {"regex_pattern": r"(0x[0-9a-fA-F]+)", "mask_with": "<HEX>"},
                {"regex_pattern": r"(\b\d{5,}\b)", "mask_with": "<NUM>"},
            ]
            self._miner = TemplateMiner(config=config)
        return self._miner

    def parse(self, log_lines: list[str]) -> dict[str, Any]:
        """
        Parse log lines using Drain3 template mining.

        Args:
            log_lines: List of raw log line strings.

        Returns:
            Dictionary with templates and cluster assignments.
        """
        miner = self._get_miner()
        clusters = []
        line_clusters = []

        for i, line in enumerate(log_lines):
            line = line.strip()
            if not line:
                continue

            result = miner.add_log_message(line)
            line_clusters.append({
                "line_number": i + 1,
                "cluster_id": result.get("cluster_id", result.get("cluster_id", -1))
                if isinstance(result, dict) else getattr(result, "cluster_id", -1),
                "template": result.get("template_mined", "")
                if isinstance(result, dict) else getattr(result, "template_mined", ""),
            })
```


DeepLog LSTM neural network for sequence anomaly detection.

```
# Step 3: Define LSTM model
class DeepLogModel(nn.Module):
    def __init__(self, num_classes, hidden_size, num_layers):
        super().__init__()
        self.embedding = nn.Embedding(num_classes, hidden_size)
        self.lstm = nn.LSTM(
            hidden_size, hidden_size, num_layers, batch_first=True, dropout=0.1
        )
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        emb = self.embedding(x)
        out, _ = self.lstm(emb)
        out = self.fc(out[:, -1, :])
        return out

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = DeepLogModel(num_classes, self.hidden_size, self.num_layers).to(device)

# Step 4: Train the model (self-supervised)
dataset = TensorDataset(X, y)
loader = DataLoader(dataset, batch_size=self.batch_size, shuffle=True)
optimizer = torch.optim.Adam(model.parameters(), lr=self.learning_rate)
criterion = nn.CrossEntropyLoss()

model.train()
for epoch in range(self.num_epochs):
    total_loss = 0
    for batch_x, batch_y in loader:
        batch_x, batch_y = batch_x.to(device), batch_y.to(device)
        optimizer.zero_grad()
        output = model(batch_x)
        loss = criterion(output, batch_y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

# Step 5: Detect anomalies (inference)
model.eval()
anomalies = []
with torch.no_grad():
    for i in range(len(sequences)):
        seq_tensor = torch.tensor([sequences[i]], dtype=torch.long).to(device)
        output = model(seq_tensor)
        probs = torch.softmax(output, dim=1)
        top_k_values, top_k_indices = torch.topk(probs, min(self.top_k, num_classes))

        actual = labels[i]
        predicted_set = top_k_indices[0].tolist()

        if actual not in predicted_set:
            line_idx = i + self.window_size
            confidence = 1.0 - probs[0][actual].item()
            anomalies.append({
                "line_number": line_idx + 1,
                "content": log_lines[line_idx].strip()[:200],
                "actual_template_id": actual,
                "predicted_template_ids": predicted_set[:5],
                "confidence": round(confidence, 4),
                "anomaly_type": "sequence_deviation",
            })
```

PyOD Isolation Forest for statistical outlier detection.

```
def detect(self, log_lines: list[str]) -> dict[str, Any]:
    """
    Run anomaly detection on log lines.

    Args:
        log_lines: List of raw log strings.

    Returns:
        Dictionary with anomaly results.
    """
    features = self._extract_features(log_lines)
    if len(features) < 20:
        return {
            "engine": "pyod",
            "anomalies": [],
            "message": "Too few log lines for statistical anomaly detection (need >= 20)",
        }

    try:
        import numpy as np
        from pyod.models.iforest import IForest

        feature_names = ["line_length", "word_count", "level_numeric", "has_ip", "has_exception", "digit_ratio", "special_char_count"]
        X = np.array([[f[k] for k in feature_names] for f in features])

        clf = IForest(contamination=self.contamination, random_state=42)
        clf.fit(X)

        labels = clf.labels_
        scores = clf.decision_scores_

        anomalies = []
        for i, (label, score) in enumerate(zip(labels, scores)):
            if label == 1:
                anomalies.append({
                    "line_number": i + 1,
                    "anomaly_score": round(float(score), 4),
                    "content": log_lines[i].strip()[:200],
                    "features": features[i],
                })

        anomalies.sort(key=lambda x: x["anomaly_score"], reverse=True)

        return {
            "engine": "pyod",
            "anomalies": anomalies[:50],
            "total_anomalies": len(anomalies),
            "total_lines_analyzed": len(features),
            "anomaly_rate": round(len(anomalies) / len(features) * 100, 2),
        }

    except ImportError:
        return {
            "engine": "pyod",
            "anomalies": [],
            "error": "PyOD not installed",
        }
    except Exception as e:
        return {
            "engine": "pyod",
            "anomalies": [],
            "error": str(e),
        }
```

FlashText keyword scanning with 100+ terms.

```
# Keyword categories with severity mapping
KEYWORD_CATEGORIES = {
    "critical": {
        "severity": "critical",
        "keywords": [
            "FATAL", "PANIC", "EMERGENCY", "EMERG", "CRITICAL",
            "SEVERE", "unrecoverable", "system crash", "kernel panic",
            "out of memory", "OOM", "data corruption", "data loss",
        ],
    },
    "error": {
        "severity": "error",
        "keywords": [
            "ERROR", "Exception", "FAILURE", "FAILED", "exception",
            "traceback", "stack trace", "NullPointerException",
            "IOException", "RuntimeException", "PermissionError",
            "ConnectionRefused", "Connection refused", "Connection reset",
            "Broken pipe", "EOFException", "timeout", "timed out",
            "segfault", "core dump", "abort",
        ],
    },
    "warning": {
        "severity": "warning",
        "keywords": [
            "WARNING", "WARN", "DEPRECATED", "deprecated",
            "Redundant", "retry", "retrying", "slow",
            "high latency", "disk full", "low memory",
            "certificate expir", "insecure", "not recommended",
        ],
    },
    "security": {
        "severity": "error",
        "keywords": [
            "unauthorized", "denied", "forbidden", "authentication failure",
            "Failed password", "invalid user", "illegal user",
            "BREAK-IN ATTEMPT", "injection", "XSS",
            "brute force", "Too many authentication failures",
            "SYN flooding",
        ],
    },
    "performance": {
        "severity": "warning",
        "keywords": [
            "timeout", "latency", "slow", "thrott",
            "queue full", "backpressure", "congestion",
            "memory leak", "swap", "high cpu", "deadlock",
        ],
    },
}

class KeywordDetector:
    """Detect error keywords in log lines using FlashText."""

    def __init__(self):
        self._processor = None
        self._keyword_severity: dict[str, str] = {}

    def _init_processor(self):
        """Lazy-initialize the FlashText KeywordProcessor."""
        if self._processor is not None:
            return

        from flashtext import KeywordProcessor

        self._processor = KeywordProcessor(case_sensitive=False)

        for category, config in KEYWORD_CATEGORIES.items():
            severity = config["severity"]
            for kw in config["keywords"]:
                clean_name = f"{category}::{kw}"
                self._processor.add_keyword(kw, clean_name)
                self._keyword_severity[clean_name] = severity
```

Activity 2.3: Event Correlation and Predictive Analysis Tools correlate_events:

Detects temporal clustering, root causes, cascading failures, and evaluates impact of components

```
# Step 1: Parse all lines into structured events
events = []
for i, line in enumerate(log_lines):
    line = line.strip()
    if not line:
        continue

    severity = _extract_severity(line)
    if severity in ("info", "debug", "trace"):
        continue # Only correlate errors/warnings

    events.append({
        "line_number": i + 1,
        "timestamp": _extract_timestamp(line),
        "component": _extract_component(line),
        "severity": severity,
        "content": line[:300],
    })

if not events:
    return json.dumps({
        "event_groups": [],
        "timeline": [],
        "root_causes": [],
        "total_correlated_events": 0,
        "message": "No error/warning events found to correlate",
    })

# Step 2: Group events by temporal proximity
event_groups = []
current_group = [events[0]]

for event in events[1:]:
    # If timestamps are available, use time-based grouping
    # Otherwise, use line-proximity (within 10 lines)
    if current_group:
        last = current_group[-1]
        line_gap = event["line_number"] - last["line_number"]
        if line_gap <= 10:
            current_group.append(event)
        else:
            event_groups.append(current_group)
            current_group = [event]
    else:
        current_group = [event]

if current_group:
    event_groups.append(current_group)

# Step 4: Build timeline
timeline = []
for group in event_groups[:20]:
    timeline.append({
        "line_start": group[0]["line_number"],
        "line_end": group[-1]["line_number"],
        "event_count": len(group),
        "primary_component": group[0]["component"],
        "max_severity": max(
            (e["severity"] for e in group),
            key=lambda s: {"critical": 4, "fatal": 4, "emergency": 4,
                          "error": 3, "warning": 2}.get(s, 1),
        ),
        "summary": group[0]["content"][:100],
    })

# Step 5: Component impact analysis
component_impact = [
    {"component": comp, "error_count": count, "impact_rank": rank + 1}
    for rank, (comp, count) in enumerate(
        sorted(component_error_counts.items(), key=lambda x: -x[1])
    )
]

return json.dumps({
    "total_error_events": len(events),
    "total_event_groups": len(event_groups),
    "cascading_failures": cascading_failures[:10],
    "root_cause_candidates": root_causes[:10],
    "timeline": timeline,
    "component_impact": component_impact[:15],
    "summary": {
        "cascading_failure_count": len(cascading_failures),
        "unique_components_affected": len(component_error_counts),
        "most_affected_component": (
            component_impact[0]["component"] if component_impact else "N/A"
        ),
    },
}, default=str)
```

`predict_failures(log_content, analysis_depth="standard")`: Analyzes error trends and generates per-component risk scoring.

```
num_segments = min(10, max(3, total_lines // 50))
segment_size = total_lines // num_segments
segment_counts = [0] * num_segments

for event in events:
    seg_idx = min((event["line_number"] - 1) // segment_size, num_segments - 1)
    segment_counts[seg_idx] += 1

# Linear regression on segment error counts
x_vals = list(range(num_segments))
slope, intercept, r_sq = _linear_regression(
    [float(x) for x in x_vals],
    [float(c) for c in segment_counts],
)

# Determine trend direction
if slope > 0.5 and r_sq > 0.3:
    trend = "INCREASING"
    trend_description = f"Error rate is increasing by ~{abs(slope):.1f} errors per segment"
elif slope < -0.5 and r_sq > 0.3:
    trend = "DECREASING"
    trend_description = f"Error rate is decreasing by ~{abs(slope):.1f} errors per segment"
else:
    trend = "STABLE"
    trend_description = "Error rate is relatively stable across the log"

# — Per-Component Analysis —
component_events: dict[str, list[int]] = defaultdict(list)
for event in events:
    component_events[event["component"]].append(event["line_number"])

component_predictions = []
for comp, positions in component_events.items():
    count = len(positions)
    rate = count / total_lines * 100
    comp_segments = [0] * num_segments
    for pos in positions:
        seg_idx = min((pos - 1) // segment_size, num_segments - 1)
        comp_segments[seg_idx] += 1

    comp_slope, _, comp_r2 = _linear_regression(
        [float(x) for x in x_vals],
        [float(c) for c in comp_segments],
    )

    # Periodicity check
    periodicity = _detect_periodicity(positions)

    # Risk assessment
    if comp_slope > 1.0 and comp_r2 > 0.4:
        risk = "HIGH"
        prediction = f"Component '{comp}' error rate is accelerating – likely failure"
    elif comp_slope > 0.3:
        risk = "MEDIUM"
        prediction = f"Component '{comp}' shows gradually increasing errors"
    elif periodicity and periodicity.get("is_periodic"):
        risk = "MEDIUM"
        prediction = f"Component '{comp}' has recurring errors ({periodicity['pattern']})"
    else:
        risk = "LOW"
        prediction = f"Component '{comp}' error rate is stable"

    component_predictions.append({
        "component": comp,
        "error_count": count,
        "error_rate_pct": round(rate, 2),
        "trend_slope": round(comp_slope, 3),
        "trend_r_squared": round(comp_r2, 3),
        "risk_level": risk,
        "prediction": prediction,
        "is_periodic": bool(periodicity),
        "periodicity": periodicity,
    })
```

Activity 2.4: Auditing and Reporting tool audit_and_generate_report

Reports compliance with SOC2, HIPAA, PCI-DSS, ISO 27001, OWASP and GDPR. Identifies PII, assesses audit log completeness, calculates compliance scores.

```
# Security compliance patterns
SECURITY_CHECKS = {
    "unencrypted_connection": {
        "pattern": re.compile(r"(?http://|ftp://|telnet://|plain\s*text|unencrypted)", re.I),
        "severity": "high",
        "standard": "SOC2/HIPAA",
        "description": "Unencrypted connection or protocol detected",
    },
    "auth_failure_burst": {
        "pattern": re.compile(r"(?authentication fail|failed password|invalid user|access denied|login fail)", re.I),
        "severity": "high",
        "standard": "SOC2",
        "description": "Authentication failure – possible brute force attempt",
    },
    "privilege_escalation": {
        "pattern": re.compile(r"(?sudo|su\s|privilege|root\s+access|admin\s+login|escalat)", re.I),
        "severity": "medium",
        "standard": "SOC2/ISO27001",
        "description": "Privilege escalation or elevated access event",
    },
    "cert_issue": {
        "pattern": re.compile(r"(?certificate\s+(?:expired|invalid|error|warning)|SSL\s+error|TLS\s+handshake\s+fail)", re.I),
        "severity": "high",
        "standard": "PCI-DSS",
        "description": "Certificate or TLS/SSL issue detected",
    },
    "data_exposure": {
        "pattern": re.compile(r"(?stack\s*trace|debug\s*mode|verbose|dump|core\s*dump)", re.I),
        "severity": "medium",
        "standard": "OWASP",
        "description": "Potential data exposure through debug output or stack traces",
    },
    "service_account_misuse": {
        "pattern": re.compile(r"(?service\s+account|system\s+user|machine\s+identity).*(?:fail|deny|reject)", re.I),
        "severity": "high",
        "standard": "SOC2",
        "description": "Service account or system identity issue",
    },
}

# PII detection patterns
PII_PATTERNS = {
    "email": re.compile(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"),
    "ipv4_address": re.compile(r"\b(?:\d{1,3}\.){3}\d{1,3}\b"),
    "ssn": re.compile(r"\b\d{3}-\d{2}-\d{4}\b"),
    "credit_card": re.compile(r"\b(?:\d{4}[-\s]?){3}\d{4}\b"),
    "phone_number": re.compile(r"\b(?:\+?\d{1,3}[-.\s]?)(?(\d{3}\)\)?[-.\s]?)\d{3}[-.\s]?)\d{4}\b"),
    "api_key_pattern": re.compile(r"(?api[_-]?key|token|secret|password|passwd)\s*[=:]\s*\S+", re.I),
}
```

MILESTONE 3: Backend: FastAPI & LLM In this milestone, the FastAPI backend of LogInsightAI is developed. This serves as the intermediary between the user interface and the AI processing engine powered by Gemini, using the MCP protocol. It has a modular structure which offers maintainable API routes for log management, full AI analysis, parse-only analysis and detection-only analysis. It includes Cross-Origin Request Setup (CORS), Lifespan for server start/stop, and service modules for MCP client and LLM interaction.

Activity 3.1: FastAPI Endpoints Log Management Routes :

Routes for uploading, listing, getting, deleting logs, and loading sample logs.

```
@router.post("/upload", response_model=LogUploadResponse)
async def upload_log(file: UploadFile = File(...)):
    """Upload a log file for analysis."""
    if not file.filename:
        raise HTTPException(400, "No filename provided")

    content = await file.read()
    if len(content) > 10 * 1024 * 1024:
        raise HTTPException(413, "File too large (max 10 MB)")

    text = content.decode("utf-8", errors="replace")
    lines = text.strip().split("\n")

    log_id = str(uuid.uuid4())[:8]
    file_path = UPLOAD_DIR / f"{log_id}_{file.filename}"

    with open(file_path, "w", encoding="utf-8") as f:
        f.write(text)

    uploaded_logs[log_id] = {
        "filename": file.filename,
        "path": str(file_path),
        "line_count": len(lines),
        "size_bytes": len(content),
    }

    return LogUploadResponse(
        log_id=log_id,
        filename=file.filename,
        line_count=len(lines),
        size_bytes=len(content),
    )
```

Analysis Routes (/analyze): Trigger full AI analysis, parse-only, and detect-only workflows.

```
@router.post("/analyze", response_model=AnalysisResponse)
async def analyze_log(request: AnalysisRequest):
    """
    Run full analysis pipeline on a log file.

    analysis_type options:
    - "full": Parse + Detect (5 engines) + Correlate + Predict + Compliance + AI Summary
    - "parse": Parse only (no LLM, no detection)
    - "detect": Detect only (5 engines, no LLM, no parsing)
    """
    try:
        log_content = get_log_content(request.log_id)
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(404, f"Could not read log: {str(e)}")

    try:
        result = await _orchestrator(
            log_content=log_content,
            analysis_type=request.analysis_type,
            sensitivity=request.sensitivity,
            log_format=request.log_format,
        )

        return AnalysisResponse(
            log_id=request.log_id,
            analysis_type=request.analysis_type,
            parse_result=result.get("parse_result"),
            detection_result=result.get("detection_result"),
            correlation_result=result.get("correlation_result"),
            prediction_result=result.get("prediction_result"),
            compliance_result=result.get("compliance_result"),
            ai_summary=result.get("ai_summary"),
            engines_used=result.get("engines_used", []),
            timing=result.get("timing", {}),
            status="completed",
        )

    except Exception as e:
        logger.error(f"Analysis failed: {e}", exc_info=True)
        return AnalysisResponse(
            log_id=request.log_id,
            analysis_type=request.analysis_type,
            status="error",
            error=str(e),
        )
```


Activity 3.2: LLM Orchestrator

Agentic Tool Calling The LLM Orchestrator adopts an agentic approach with Gemini

3.1 lite preview Flash determining the sequence of MCP tools to use

```
def _get_tool_declarations(self):
    """Define tool schemas for Gemini function calling (5 tools)."""
    from google.genai import types

    parse_logs_tool = types.FunctionDeclaration(
        name="parse_logs",
        description="Parse system log content to extract structured information, templates, and statistics. Uses Drain3 template mining, Grok patterns, JSON parsing, and other tools.",
        parameters=types.Schema(
            type="OBJECT",
            properties={
                "log_content": types.Schema(
                    type="STRING",
                    description="The raw log file content as a string.",
                ),
                "log_format": types.Schema(
                    type="STRING",
                    description="Expected log format: 'auto', 'syslog', 'apache_access', 'apache_error', 'hdfs', 'zookeeper', 'json'.",
                ),
            },
            required=["log_content"],
        ),
    )

    detect_errors_tool = types.FunctionDeclaration(
        name="detect_errors",
        description="Detect errors, anomalies, and security issues using 5 engines: LogAI ML, FlashText keywords, PyOD Isolation Forest, Regex patterns, and DeepLog.",
        parameters=types.Schema(
            type="OBJECT",
            properties={
                "log_content": types.Schema(
                    type="STRING",
                    description="The raw log file content as a string.",
                ),
                "sensitivity": types.Schema(
                    type="STRING",
                    description="Detection sensitivity: 'low', 'medium', or 'high'.",
                ),
            },
        ),
    )
```

MCP Client communicates with the MCP Server via HTTP/SSE to execute tool calls.

```
class MCPClient:
    """MCP client using the official SDK with SSE transport."""

    def __init__(self, server_url: str = "http://127.0.0.1:8001"):
        self.server_url = server_url.rstrip("/")
        self._sse_url = f"{self.server_url}/sse"

    @asynccontextmanager
    async def _get_session(self):
        """Create an MCP client session via SSE transport."""
        async with sse_client(url=self._sse_url) as (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()
                yield session

    async def call_tool(self, tool_name: str, arguments: dict[str, Any]) -> dict[str, Any]:
        """
        Call an MCP tool via SSE transport using the official SDK.

        Args:
            tool_name: Name of the tool to call.
            arguments: Dictionary of arguments.

        Returns:
            Tool execution result as a dictionary.

        Raises:
            Exception: If the MCP server is unreachable or returns an error.
        """
        logger.info(f"Calling MCP tool: {tool_name}")

        async with self._get_session() as session:
            result = await session.call_tool(tool_name, arguments=arguments)

            # Extract text content from the MCP result
            if result.content:
                for block in result.content:
                    if hasattr(block, "text") and block.text:
                        return json.loads(block.text)

            return {"error": "MCP tool returned no content"}

    async def health_check(self) -> bool:
        """Check if the MCP server is accessible."""
        try:
            async with self._get_session() as session:
                # If we can initialize a session, the server is healthy
                return True
        except Exception:
            return False

    async def close(self):
        """Cleanup (no persistent state to close with per-call sessions)."""
        pass
```

Activity 3.3: Pydantic Models and Request/Response Schemas

Create request and response models with Pydantic for type checking and validation.

```
class LogUploadResponse(BaseModel):
    """Response after uploading a log file."""
    log_id: str
    filename: str
    line_count: int
    size_bytes: int
    message: str = "File uploaded successfully"

class SampleLogInfo(BaseModel):
    """Info about a sample log file."""
    name: str
    display_name: str
    size_bytes: int
    line_count: int

class AnalysisRequest(BaseModel):
    """Request to analyze a log file."""
    log_id: str
    analysis_type: str = Field(default="full", pattern="^(full|parse|detect)$")
    sensitivity: str = Field(default="medium", pattern="^(low|medium|high)$")
    log_format: str = Field(default="auto")

class AnalysisResponse(BaseModel):
    """Full analysis response v2.0."""
    analysis_id: str = Field(default_factory=lambda: str(uuid.uuid4())[:12])
    timestamp: str = Field(default_factory=lambda: datetime.now().isoformat())
    log_id: str
    analysis_type: str
    parse_result: Optional[dict] = None
    detection_result: Optional[dict] = None
    correlation_result: Optional[dict] = None
    prediction_result: Optional[dict] = None
    compliance_result: Optional[dict] = None
    ai_summary: Optional[str] = None
    engines_used: list[str] = Field(default_factory=list)
    timing: dict[str, float] = Field(default_factory=dict)
    status: str = "completed"
    error: Optional[str] = None

class CompareRequest(BaseModel):
    """Request to compare two log analyses."""
    log_id_a: str
    log_id_b: str
    analysis_type: str = Field(default="full", pattern="^(full|parse|detect)$")
    sensitivity: str = Field(default="medium", pattern="^(low|medium|high)$")
```

Activity 3.4: Server Startup and Entry Point

FastAPI application startup and entry point with Uvicorn server, CORS, and static files.

```
# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include API routers
app.include_router(logs.router)
app.include_router(analysis.router)

# Serve frontend static files
if FRONTEND_DIR.exists():
    app.mount("/", StaticFiles(directory=str(FRONTEND_DIR), html=True), name="frontend")

@app.get("/api/health")
async def health():
    """Global health check."""
    return {"status": "healthy", "service": "LogInsight AI", "version": "1.0.0"}

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
)
logger = logging.getLogger("loginsight")

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifecycle events."""
    logger.info("🚀 LogInsight AI Backend starting...")
    logger.info(f"📁 Upload directory: {UPLOAD_DIR}")
    logger.info(f"🌐 Frontend served from: {FRONTEND_DIR}")

    # Check MCP server connectivity
    from backend.routes.analysis import _mcp_client
    mcp_ok = await _mcp_client.health_check()
    if mcp_ok:
        logger.info("✅ MCP server connected")
    else:
        logger.warning("⚠️ MCP server not reachable – using direct tool execution fallback")

    yield

    # Cleanup
    await _mcp_client.close()
    logger.info("🛑 LogInsight AI Backend shutting down")

# Create FastAPI app
app = FastAPI(
    title="LogInsight AI",
    description="Intelligent Log Analyzer – Enterprise IT log analysis powered by Gemini Vertex AI",
    version="1.0.0",
    lifespan=lifespan,
)
```

Starting both servers:

- Terminal 1 — MCP Server: `cd mcp_server && python server.py`
- Terminal 2 — Backend: `python -m uvicorn backend.main:app --reload`

```
C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main>cd mcp_server
C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main\mcp_server>python server.py
[MCP] LogInsight MCP Server starting on http://127.0.0.1:8001
[MCP] Registered 5 tools: parse_logs, detect_errors, correlate_events, predict_failures, generate_report
INFO: Started server process [20356]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO: 127.0.0.1:64279 - "GET /sse HTTP/1.1" 200 OK
INFO: 127.0.0.1:64280 - "POST /messages/?session_id=2a157f77164140779c75e324f7bef676 HTTP/1.1" 202 Accepted

C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main>cd "GenAI 2 Project"
The system cannot find the path specified.

C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main>python -m uvicorn backend.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Akhil\\Desktop\\LogInsightAI-AI-Log-Analyzer-main']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [10612] using StatReload
INFO: Started server process [5380]
INFO: Waiting for application startup.
2026-04-28 15:31:53,879 [loginsight] INFO: 🚀 LogInsight AI Backend starting...
2026-04-28 15:31:53,880 [loginsight] INFO: 📁 Upload directory: C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main\uploads
2026-04-28 15:31:53,881 [loginsight] INFO: 🌐 Frontend served from: C:\Users\Akhil\Desktop\LogInsightAI-AI-Log-Analyzer-main\frontend
2026-04-28 15:31:55,014 [httpx] INFO: HTTP Request: GET http://127.0.0.1:8001/sse "HTTP/1.1 200 OK"
2026-04-28 15:31:55,038 [httpx] INFO: HTTP Request: POST http://127.0.0.1:8001/messages/?session_id=ed1ed41181304f74851e58653cdf77e7 "HTTP/1.1 202 Accepted"
2026-04-28 15:31:55,054 [loginsight] INFO: ✅ MCP server connected
INFO: Application startup complete.
```

MILESTONE 4: UI Development

This milestone focuses on developing a lightweight, responsive, and visually stunning frontend for LogInsightAI using Vanilla JavaScript, HTML, and CSS. The interface enables users to interact seamlessly with the system's log upload, multi-engine analysis, and AI-powered reporting capabilities. It is tightly integrated with FastAPI backend endpoints to facilitate real-time file management and analysis result rendering. The design follows a "Clinical Precision" aesthetic inspired by Linear.app, Vercel, Stripe, and Datadog—featuring a sleek dark theme, DM Mono code typography, and dynamic animations.

Activity 4.1: Develop HTML Frontend Interface

Design a single-page application with two views: Landing Page and Dashboard.

```
Ctrl+F for Agent
# Serve frontend static files
if FRONTEND_DIR.exists():
    app.mount("/", StaticFiles(directory=str(FRONTEND_DIR), html=True), name="frontend")
```

Templates and component structure:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="LogInsight AI- Enterprise intelligent log analysis powered by Gemini AI and MCP">
  <title>LogInsight AI</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Outfit:wght@300;400;500;600;700;800&family=DM+Mono:wght@300;400;500&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="css/styles.css?v=30">
  <link rel="stylesheet" href="css/dashboard.css?v=30">
  <link rel="stylesheet" href="https://unpkg.com/aos@2.3.4/dist/aos.css">
  <script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r134/three.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/vanta@latest/dist/vanta.waves.min.js"></script>
  <script src="https://unpkg.com/aos@2.3.4/dist/aos.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/typed.js@2.0.16/dist/typed.umd.js"></script>
  <script type="module" src="https://cdn.jsdelivr.net/npm/lidrs/dist/auto/quantum.js"></script>
  <link rel="icon"
    href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'><rect width='32' height='32' rx='8' fill='%234F46E5' /><path d='M8 22V10 4 4 6 4 6 4 4v12z' fill='%23fff' opacity='.9' /></svg>"
    />
</head>
```

Activity 4.2: CSS Design System — Clinical Precision

Base design system with CSS Custom Properties for consistent theming.

```
✓ #page-dashboard .sidebar-item {  
  font-family: var(--font-ui);  
  font-size: .82rem;  
  color: var(--text-secondary);  
  border-radius: var(--radius-sm);  
  padding: 8px 12px;  
  transition: all var(--transition);  
}  
  
✓ #page-dashboard .sidebar-item:hover {  
  background: var(--bg-sunken);  
  color: var(--text-primary);  
}  
  
✓ #page-dashboard .sidebar-item.active {  
  background: var(--accent-light);  
  color: var(--accent);  
  font-weight: 600;  
}  
  
✓ #page-dashboard .sidebar-item.active::before {  
  display: none;  
}  
  
✓ #page-dashboard .sidebar-item.active svg {  
  color: var(--accent);  
}  
  
✓ #page-dashboard .sidebar-action-btn.primary-action {  
  background: var(--accent);  
  color: ■ #fff;  
  font-family: var(--font-ui);  
  font-weight: 600;  
  font-size: .82rem;  
  border-radius: var(--radius-sm);  
  box-shadow: 0 1px 3px ■ rgba(26, 86, 219, 0.2);  
  padding: 9px 14px;  
  transition: all var(--transition);  
}
```

Activity 4.3: JavaScript UI Controller

File upload handler with drag-and-drop support and progress indicators.

```
class UI {
  constructor() {
    this.page = 'landing';
    this.tab = 'analysis';

    document.querySelectorAll('.nav-link[data-page]').forEach(btn =>
      btn.addEventListener('click', () => this.showPage(btn.dataset.page))
    );
    document.querySelectorAll('.tab-btn-dash').forEach(btn =>
      btn.addEventListener('click', () => this.switchTab(btn.dataset.tab))
    );
    document.querySelectorAll('.sidebar-item[data-tab]').forEach(btn =>
      btn.addEventListener('click', () => this.switchTab(btn.dataset.tab))
    );
    const sidebarToggle = document.getElementById('sidebar-toggle');
    const sidebar = document.getElementById('sidebar');
    if (sidebarToggle && sidebar) {
      sidebarToggle.addEventListener('click', () => sidebar.classList.toggle('collapsed'));
    }
    window.addEventListener('scroll', () => {
      document.querySelector('.glass-nav').classList.toggle('scrolled', window.scrollY > 20);
    });
  }

  showPage(page) {
    this.page = page;
    document.querySelectorAll('.page').forEach(el =>
      el.classList.toggle('active', el.id === 'page-' + page)
    );
    document.querySelectorAll('.nav-link[data-page]').forEach(btn =>
      btn.classList.toggle('active', btn.dataset.page === page)
    );
    const vantaBg = document.getElementById('vanta-bg');
    if (vantaBg) vantaBg.style.display = page === 'landing' ? '' : 'none';
    window.scrollTo(0, 0);
  }

  switchTab(tab) {
    this.tab = tab;
    document.querySelectorAll('.tab-btn-dash').forEach(btn =>
      btn.classList.toggle('active', btn.dataset.tab === tab)
    );
    document.querySelectorAll('.sidebar-item[data-tab]').forEach(btn =>
      btn.classList.toggle('active', btn.dataset.tab === tab)
    );
    document.querySelectorAll('.tab-pane').forEach(pane =>
      pane.classList.toggle('active', pane.id === 'pane-' + tab)
    );
  }
}
```


MILESTONE 5:

Testing & Optimization This involves the testing of the accuracy, effectiveness, and contextual understanding of LogInsight AI's multi-approach analysis system. We test the system with 16 pre-bundled sample log files in various formats (Apache, HDFS, Linux, OpenSSH, Windows, Zookeeper) and LogHub benchmark data (2K lines) for similar-sized log files. Optimisation work involves adjusting sensitivity, LSTM hyperparameters, Isolation Forest contamination parameters, and simplify UI/UX of the dashboard..

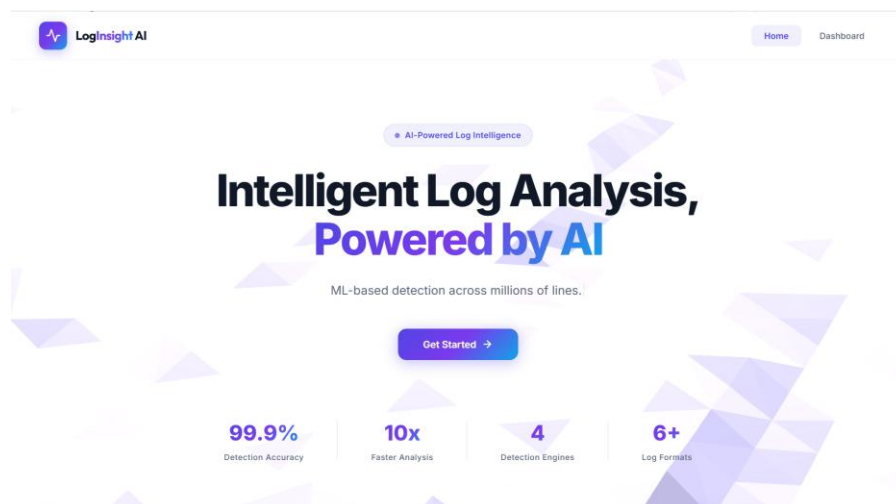
Activity 5.1:

Use Real Log Files in a Variety of Formats End-to-end test using different types of logs:

- Apache access and error logs from web servers
- HDFS logs from Hadoop file systems
- Linux system logs from processes
- OpenSSH authentication logs with security events
- Windows Event Log exports
- Zookeeper coordination service logs Landing Page

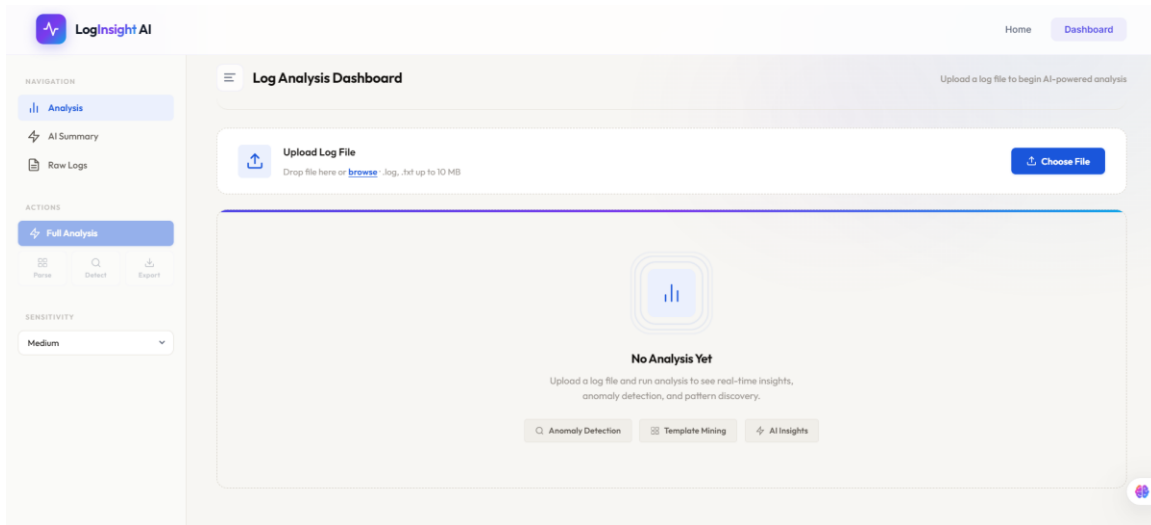
Landing Page:

Home page with an animated background via Vanta.js, hero area with Typed.js text animation, 5 feature cards showing the analysis options, and stats row.



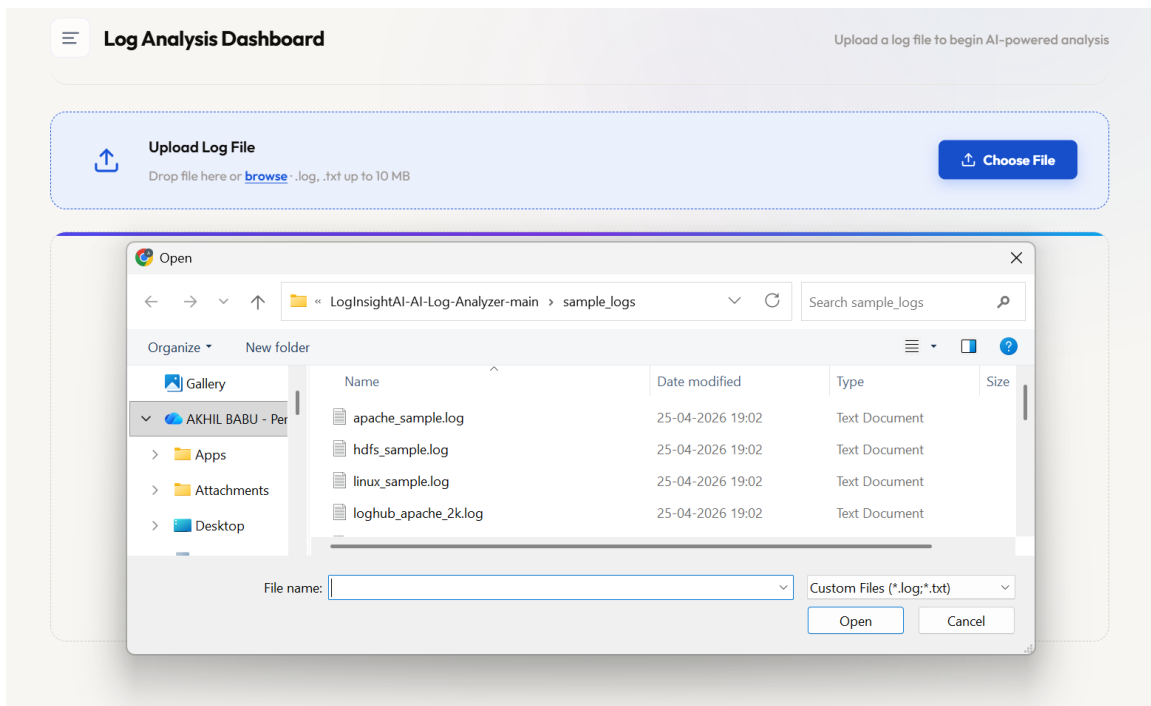
Dashboard Page:

Dashboard: The Clinical Precision analysis dashboard including the sidebar navigation, upload zone, stats grid and analysis controls..



Upload and File Management

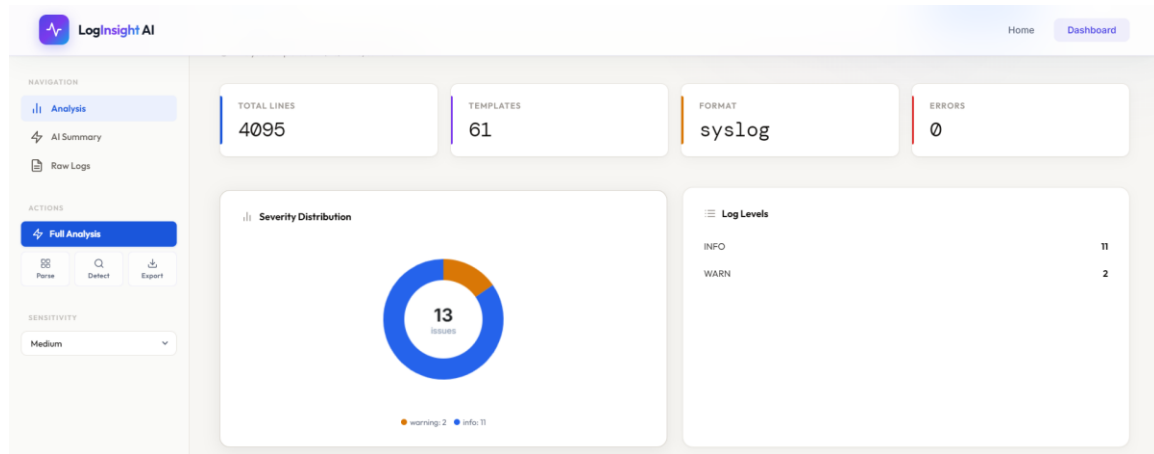
Upload Zone: File upload zone with drag-and-drop or click-to-browse, select sample log file.



Analysis Results

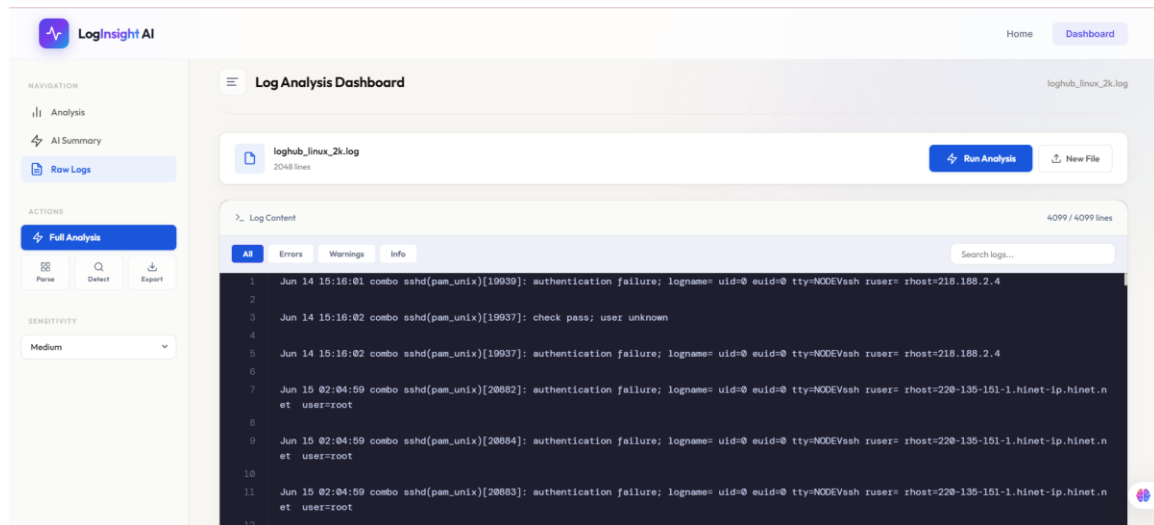
Stats Grid: Collection of metric cards showing DM Mono numbers for total lines, errors, percentage of anomalies, and health status.

Severity Chart: Canvas donut chart showing severity breakdown of issues.



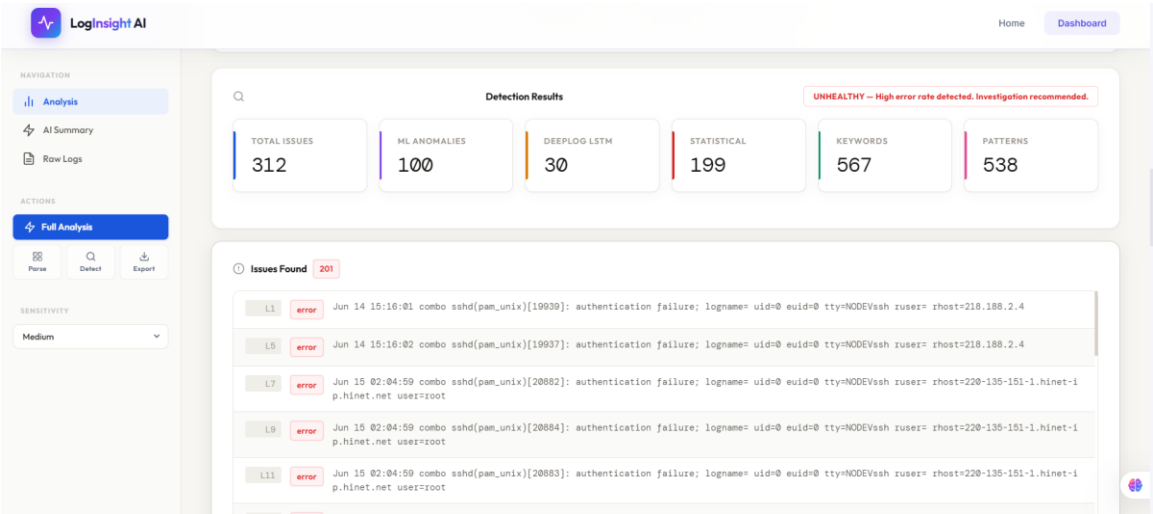
Log Viewer

Log Viewer: Interactive raw log viewer with filters, search, syntax highlighting and line numbers.



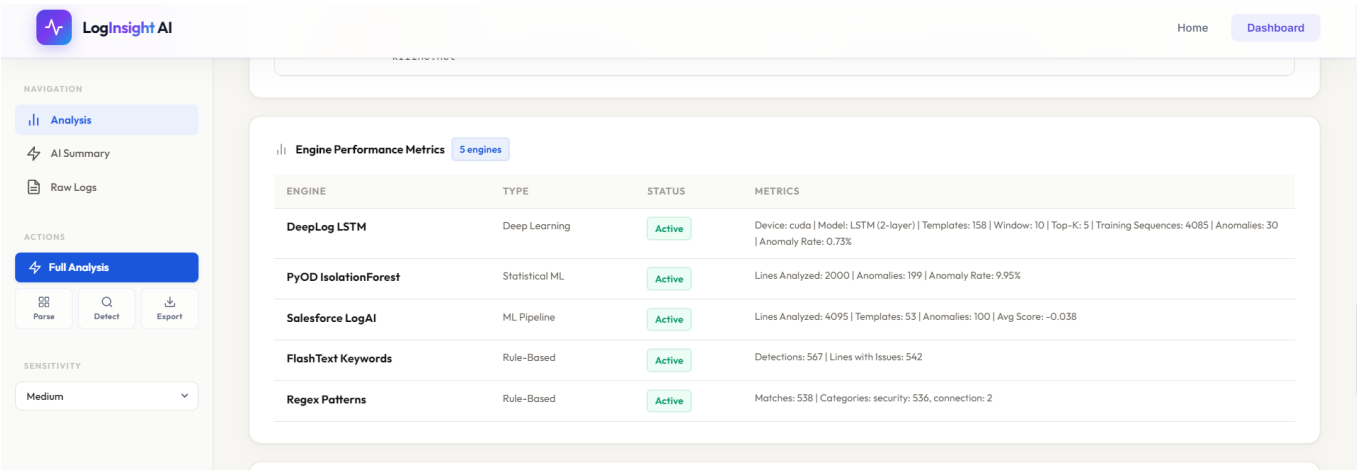
Issues Table

Issues Table: Table of detected issues with severity badges, line numbers and engine.



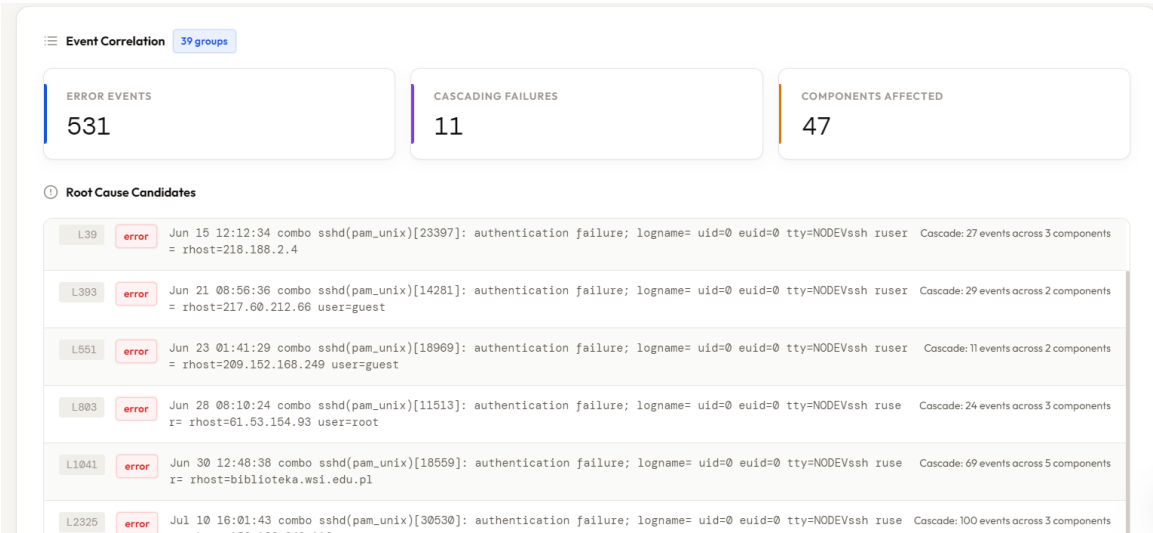
Engine Metrics

Engine Performance: Table displaying the results of 5 detection engines, number of detections, and time taken



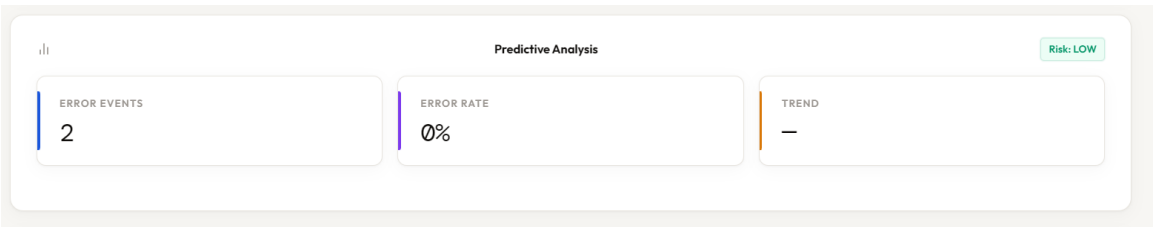
Event Correlation Results

Correlation Panel: Root cause suspects, failure chains, component impact rank.



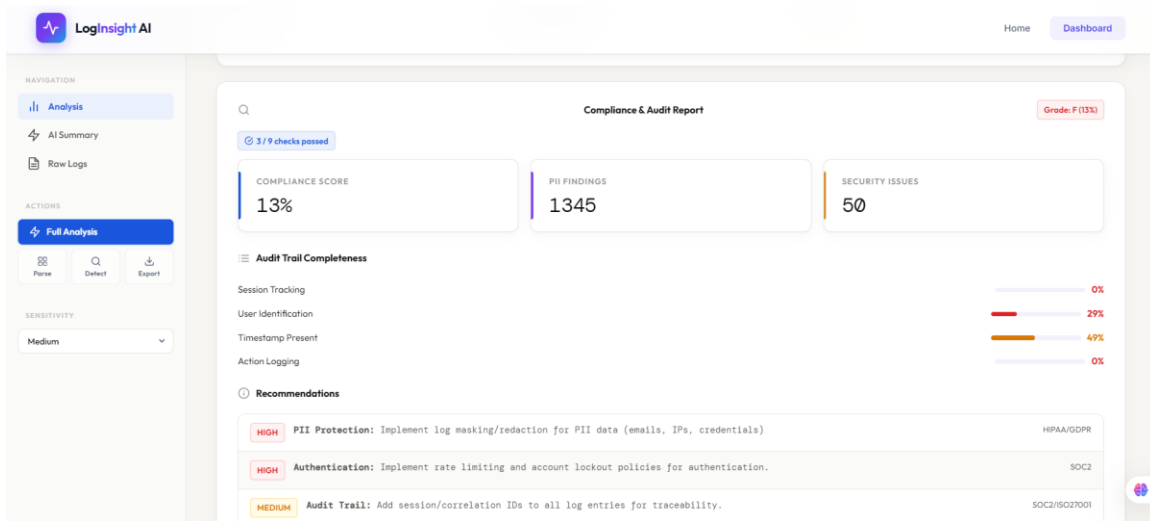
Predictive Failure Analysis

Prediction Panel: Probability badge, predictions for each component, error rate trends



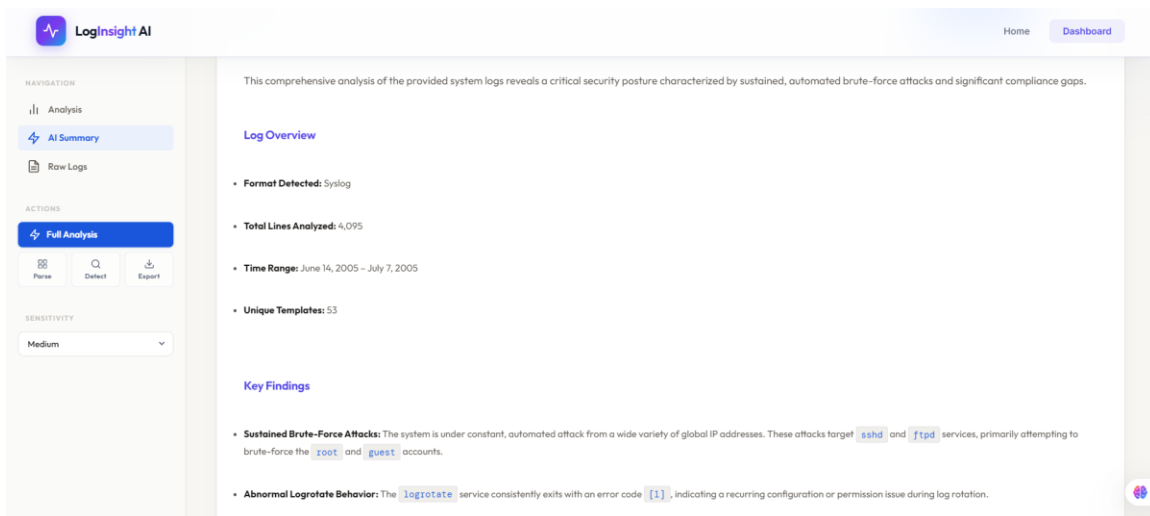
Compliance Audit Results

Compliance Panel: Grading badge (A-F), audit trail bars, PII issues, per framework results



AI Summary (Gemini Analysis)

AI Summary: Markdown-formatted report of Gemini 3.1 lite preview Flash with overview, summary, anomaly, correlation, prediction, and recommendations.



CONCLUSION

LogInsightAI is a smart and scalable log management system that uses AI-driven multi-engine analysis for anomaly detection, root cause analysis and compliance auditing. Harnessing the power of FastAPI and Gemini 3.1 lite preview Flash via the Model Context Protocol (MCP), the system intelligently orchestrates 5 tools for analysis - log parsing, error detection, correlation, failure prediction and compliance reporting - through an agentic workflow driven by the Large Language Model (LLM) that orchestrates the strategy for analysis. LogInsightAI has been designed for enterprise-level resilience, scalability and analysis capabilities in its FastAPI backend, AI integration and responsive frontend. FastAPI's async routing and service design allow for optimal performance and maintainability, with the MCP Server exposing tools via the Model Context Protocol. The Vanilla JavaScript frontend provides a user-friendly Clinical Precision dashboard similar to industry giants such as Linear.app, Vercel and Datadog. Each engine (4-engine log parser, 5-engine anomaly detector, 6-framework compliance auditor) offers robust log analysis functionality that supports multiple log formats and applications. From root cause analysis of a production outage via cascading failure analysis to GDPR compliance checks for PII exposure to predicting component failure from error trend analysis, LogInsightAI serves as a smart analysis agent, converting logs into knowledge. Through its integration of deep learning (DeepLog LSTM), statistical ML (PyOD Isolation Forest), ML pipelines (Salesforce LogAI), rule-based detection (FlashText, Regex) and generative AI orchestration (Gemini 3.1 lite preview Flash via MCP), LogInsightAI is a real-world example of how agentic AI can improve enterprise workflows, bringing smart log analysis, predictive monitoring and compliance automation to the fingertips of teams - adaptive, comprehensive and ready for production.